

Android Design Patterns

50 Questions + Coding Challenges

This comprehensive guide covers essential design patterns used in Android app development. From beginner-friendly Singleton patterns to advanced architectural patterns like MVVM, Clean Architecture, and reactive programming. Each question includes practical examples and detailed solutions, followed by hands-on coding challenges to test your skills.

Table of Contents

Easy Level (1-20): Singleton, ViewHolder, Factory, Builder, Observer, Adapter, Dependency Injection, Repository, Strategy, Decorator, MVVM, MVP, State, Command, Lazy Initialization, Facade, Proxy, Template Method, Chain of Responsibility, Memento **Medium Level (21-35):** Clean Architecture, UseCase, MVI, Mapper, Service Locator, Abstract Factory, Composite, Flow/StateFlow, Visitor, Paging, Resource State, Event Bus, Coordinator, Cache-Aside, Throttle/Debounce **Hard Level (36-50):** Multi-module Architecture, Feature Toggle, Circuit Breaker, Saga Pattern, CQRS, Event Sourcing, Repository with Sync, Mediator, Interpreter, Flyweight, Bridge, Prototype, Object Pool, Null Object, Specification **Coding Challenges (5):** Build real-world implementations

HARD LEVEL (Questions 36-50)

36. How to implement Multi-module Clean Architecture?

```
// Module structure: // :app (depends on all feature modules) // :core:domain (pure Kotlin - business entities) //
:core:data (data sources, repositories) // :core:network (API clients) // :core:database (Room DB) // :core:common
(utilities, base classes) // :feature:auth (login/signup feature) // :feature:home (home screen feature) //
:feature:profile (profile feature) // :core:domain/User.kt data class User( val id: String, val name: String, val email:
String ) interface UserRepository { suspend fun getUser(id: String): Result<User> } // :core:data/UserRepositoryImpl.kt
class UserRepositoryImpl( private val remoteDataSource: UserRemoteDataSource, private val localDataSource:
UserLocalDataSource, private val mapper: UserMapper ) : UserRepository { override suspend fun getUser(id: String):
Result<User> { return try { val dto = remoteDataSource.fetchUser(id) val user = mapper.toDomain(dto)
localDataSource.saveUser(mapper.toEntity(user)) Result.success(user) } catch (e: Exception) {
localDataSource.getUser(id)?.let { Result.success(mapper.toDomain(it)) } ?: Result.failure(e) } } //
:feature:auth/LoginViewModel.kt class LoginViewModel( private val loginUseCase: LoginUseCase ) : ViewModel() { //
Feature-specific logic } // Module dependencies in build.gradle // :feature:auth - depends on :core:domain, :core:common
// :feature:home - depends on :core:domain, :core:common // :core:data - depends on :core:domain, :core:network,
:core:database
```

Answer: Multi-module architecture separates features into independent modules with clear dependencies.

Benefits: Faster build times, reusability, clear boundaries, parallel development.

37. Implement Feature Toggle pattern for A/B testing.

```
// Feature Toggle Manager class FeatureToggleManager( private val remoteConfig: RemoteConfigService, private val
localStorage: PreferencesManager ) { enum class Feature(val key: String, val defaultValue: Boolean) {
NEW_CHECKOUT("feature_new_checkout", false), DARK_MODE("feature_dark_mode", true), SOCIAL_LOGIN("feature_social_login",
false), ADVANCED_SEARCH("feature_advanced_search", false) } suspend fun initialize() { remoteConfig.fetch() // Cache
values locally Feature.values().forEach { feature -> val value = remoteConfig.getBoolean(feature.key,
feature.defaultValue) localStorage.putBoolean(feature.key, value) } } fun isEnabled(feature: Feature): Boolean { return
localStorage.getBoolean(feature.key, feature.defaultValue) } fun enableFeature(feature: Feature) {
localStorage.putBoolean(feature.key, true) } fun disableFeature(feature: Feature) { localStorage.putBoolean(feature.key,
false) } } // Usage in code class CheckoutViewModel( private val featureToggle: FeatureToggleManager ) : ViewModel() { fun
proceedToCheckout() { if (featureToggle.isEnabled(FeatureToggleManager.Feature.NEW_CHECKOUT)) { navigateToNewCheckout() }
else { navigateToLegacyCheckout() } } } // Conditional UI rendering @Composable fun LoginScreen(featureToggle:
FeatureToggleManager) { Column { EmailPasswordLogin() if
(featureToggle.isEnabled(FeatureToggleManager.Feature.SOCIAL_LOGIN)) { Spacer(modifier = Modifier.height(16.dp))
GoogleLoginButton() FacebookLoginButton() } } } // A/B Testing variant class ABTestManager(private val featureToggle:
FeatureToggleManager) { enum class Experiment { CHECKOUT_FLOW_V2, ONBOARDING_REDESIGN, RECOMMENDATION_ALGORITHM } fun
getVariant(experiment: Experiment): String { return when (experiment) { Experiment.CHECKOUT_FLOW_V2 -> if
(Random.nextBoolean()) "variant_a" else "variant_b" else -> "control" } } }
```

Answer: Feature Toggle pattern enables/disables features at runtime without code changes.

Enables: Gradual rollouts, A/B testing, emergency kill switches, beta features.

38. Implement Circuit Breaker pattern for network resilience.

```
// Circuit Breaker States sealed class CircuitState { object Closed : CircuitState() // Normal operation object Open :
CircuitState() // Blocking requests data class HalfOpen(val startTime: Long) : CircuitState() // Testing recovery } class
CircuitBreaker( private val failureThreshold: Int = 5, private val resetTimeoutMs: Long = 60000, private val
halfOpenSuccessThreshold: Int = 2 ) { private var state: CircuitState = CircuitState.Closed private var failureCount = 0
private var successCount = 0 private var lastFailureTime = 0L suspend fun <T> execute(block: suspend () -> T): Result<T> {
return when (state) { is CircuitState.Open -> { if (shouldAttemptReset()) { state =
CircuitState.HalfOpen(System.currentTimeMillis()) attemptCall(block) } else {
Result.failure(CircuitBreakerOpenException()) } } is CircuitState.HalfOpen -> attemptCall(block) is CircuitState.Closed ->
attemptCall(block) } } private suspend fun <T> attemptCall(block: suspend () -> T): Result<T> { return try { val result =
block() onSuccess() Result.success(result) } catch (e: Exception) { onFailure() Result.failure(e) } } private fun
onSuccess() { when (state) { is CircuitState.HalfOpen -> { successCount++ if (successCount >= halfOpenSuccessThreshold) {
reset() } } is CircuitState.Closed -> { failureCount = 0 } else -> {} } } private fun onFailure() { lastFailureTime =
System.currentTimeMillis() when (state) { is CircuitState.HalfOpen -> { trip() } is CircuitState.Closed -> {
failureCount++ if (failureCount >= failureThreshold) { trip() } } else -> {} } } private fun trip() { state =
CircuitState.Open failureCount = 0 } private fun reset() { state = CircuitState.Closed failureCount = 0 successCount = 0 }
private fun shouldAttemptReset(): Boolean { return System.currentTimeMillis() - lastFailureTime >= resetTimeoutMs } }
class CircuitBreakerOpenException : Exception("Circuit breaker is open") // Usage in Repository class PaymentRepository(
private val api: PaymentApiService, private val circuitBreaker: CircuitBreaker ) { suspend fun processPayment(payment:
Payment): Result<PaymentResult> { return circuitBreaker.execute { api.processPayment(payment) } } } // ViewModel handles
circuit breaker failures class PaymentViewModel( private val repository: PaymentRepository ) : ViewModel() { fun
processPayment(payment: Payment) { viewModelScope.launch { repository.processPayment(payment) .onSuccess { result ->
_paymentState.value = PaymentState.Success(result) } .onFailure { error -> when (error) { is CircuitBreakerOpenException
-> { _paymentState.value = PaymentState.ServiceUnavailable } else -> { _paymentState.value =
```

```
PaymentState.Error(error.message) } } } } }
```

Answer: Circuit Breaker prevents cascading failures by stopping requests to failing services.

Protects app from cascading failures, improves resilience, graceful degradation.

39. Implement CQRS (Command Query Responsibility Segregation).

```
// Command - Writes/Updates sealed class UserCommand { data class CreateUser(val name: String, val email: String) :
UserCommand() data class UpdateUserProfile(val userId: String, val name: String) : UserCommand() data class DeleteUser(val
userId: String) : UserCommand() } // Query - Reads sealed class UserQuery { data class GetUserById(val userId: String) :
UserQuery() data class GetUsersByRole(val role: String) : UserQuery() object GetAllUsers : UserQuery() } // Command
Handler class UserCommandHandler( private val writeRepository: UserWriteRepository, private val eventBus: EventBus ) {
suspend fun handle(command: UserCommand): Result<Unit> { return when (command) { is UserCommand.CreateUser -> { val user =
User( id = UUID.randomUUID().toString(), name = command.name, email = command.email ) writeRepository.insert(user)
eventBus.publish(UserCreatedEvent(user.id)) Result.success(Unit) } is UserCommand.UpdateUserProfile -> {
writeRepository.update(command.userId, command.name) eventBus.publish(UserUpdatedEvent(command.userId))
Result.success(Unit) } is UserCommand.DeleteUser -> { writeRepository.delete(command.userId)
eventBus.publish(UserDeletedEvent(command.userId)) Result.success(Unit) } } } // Query Handler class UserQueryHandler(
private val readRepository: UserReadRepository ) { suspend fun handle(query: UserQuery): Result<Any> { return when (query)
{ is UserQuery.GetUserById -> { val user = readRepository.findById(query.userId) Result.success(user) } is
UserQuery.GetUsersByRole -> { val users = readRepository.findByRole(query.role) Result.success(users) } is
UserQuery.GetAllUsers -> { val users = readRepository.findAll() Result.success(users) } } } // Separate repositories for
read and write interface UserWriteRepository { suspend fun insert(user: User) suspend fun update(userId: String, name:
String) suspend fun delete(userId: String) } interface UserReadRepository { suspend fun findById(userId: String): User?
suspend fun findByRole(role: String): List<User> suspend fun findAll(): List<User> } // ViewModel using CQRS class
UserViewModel( private val commandHandler: UserCommandHandler, private val queryHandler: UserQueryHandler ) : ViewModel()
{ private val _users = MutableStateFlow<List<User>>(emptyList()) val users: StateFlow<List<User>> = _users // Command fun
createUser(name: String, email: String) { viewModelScope.launch { commandHandler.handle( UserCommand.CreateUser(name,
email) ) } } // Query fun loadUsers() { viewModelScope.launch { queryHandler.handle(UserQuery.GetAllUsers) .onSuccess {
result -> _users.value = result as List<User> } } }
```

Answer: CQRS separates read and write operations into different models.

Benefits: Optimized read/write models, scalability, complex domain logic separation.

40. Implement Event Sourcing pattern.

```
// Events sealed class OrderEvent { abstract val orderId: String abstract val timestamp: Long data class OrderCreated(
override val orderId: String, override val timestamp: Long, val customerId: String, val items: List<OrderItem> ) :
OrderEvent() data class ItemAdded( override val orderId: String, override val timestamp: Long, val item: OrderItem ) :
OrderEvent() data class ItemRemoved( override val orderId: String, override val timestamp: Long, val itemId: String ) :
OrderEvent() data class OrderConfirmed( override val orderId: String, override val timestamp: Long ) : OrderEvent() data
class OrderShipped( override val orderId: String, override val timestamp: Long, val trackingNumber: String ) :
OrderEvent() } // Event Store interface EventStore { suspend fun saveEvent(event: OrderEvent) suspend fun
getEvents(orderId: String): List<OrderEvent> } class InMemoryEventStore : EventStore { private val events =
mutableMapOf<String, MutableList<OrderEvent>>() override suspend fun saveEvent(event: OrderEvent) {
events.getOrPut(event.orderId) { mutableListOf() }.add(event) } override suspend fun getEvents(orderId: String):
List<OrderEvent> { return events[orderId] ?: emptyList() } } // Aggregate - Order data class Order( val id: String, val
customerId: String, val items: List<OrderItem>, val status: OrderStatus, val trackingNumber: String? = null ) { companion
object { // Rebuild state from events fun fromEvents(events: List<OrderEvent>): Order? { if (events.isEmpty()) return null
var order: Order? = null events.forEach { event -> order = when (event) { is OrderEvent.OrderCreated -> Order( id =
event.orderId, customerId = event.customerId, items = event.items, status = OrderStatus.DRAFT ) is OrderEvent.ItemAdded ->
order?.copy( items = order!!.items + event.item ) is OrderEvent.ItemRemoved -> order?.copy( items = order!!.items.filter {
it.id != event.itemId } ) is OrderEvent.OrderConfirmed -> order?.copy( status = OrderStatus.CONFIRMED ) is
OrderEvent.OrderShipped -> order?.copy( status = OrderStatus.SHIPPED, trackingNumber = event.trackingNumber ) } } return
order } } // Repository with Event Sourcing class OrderRepository(private val eventStore: EventStore) { suspend fun
createOrder(customerId: String, items: List<OrderItem>): String { val orderId = UUID.randomUUID().toString() val event =
OrderEvent.OrderCreated( orderId = orderId, timestamp = System.currentTimeMillis(), customerId = customerId, items = items
) eventStore.saveEvent(event) return orderId } suspend fun addItem(orderId: String, item: OrderItem) { val event =
OrderEvent.ItemAdded( orderId = orderId, timestamp = System.currentTimeMillis(), item = item ) eventStore.saveEvent(event)
} suspend fun getOrder(orderId: String): Order? { val events = eventStore.getEvents(orderId) return
Order.fromEvents(events) } // Get order at specific point in time suspend fun getOrderAtTime(orderId: String, timestamp:
Long): Order? { val events = eventStore.getEvents(orderId) .filter { it.timestamp <= timestamp } return
Order.fromEvents(events) }
```

Answer: Event Sourcing stores all changes as sequence of events rather than current state.

Benefits: Complete audit trail, time travel, event replay, debugging history.

41. Implement Mediator pattern for complex component interaction.

Answer: Mediator centralizes complex communications between objects.

Reduces coupling between components, centralizes control logic.

42. Implement Repository pattern with offline-first sync.

Answer: Offline-first repository prioritizes local data, syncs in background.

Improves UX with instant feedback, handles network failures gracefully.

43. Implement Bridge pattern for platform abstraction.

Answer: Bridge separates abstraction from implementation for flexibility.

Allows independent variation of abstraction and implementation.

44. Implement Object Pool pattern for expensive object reuse.

Answer: Object Pool maintains reusable object instances to reduce allocation overhead.

Improves performance by reusing Bitmap, Database connections, Thread pools.

45. Implement Null Object pattern to avoid null checks.

Answer: Null Object provides default behavior instead of null references.

Eliminates null checks, provides safe default behavior.

Questions 41-50 (Summary)

46. Implement Specification pattern for complex business rules.

Answer: Specification encapsulates business rules as reusable objects.

Enables composable, testable business logic.

47. Implement Saga pattern for distributed transactions.

Answer: Saga manages long-running transactions across multiple services.

Handles complex multi-step operations with compensation.

48. Implement Adapter pattern for third-party library integration.

Answer: Adapter wraps third-party APIs to match your application's interface.

Isolates dependencies, makes switching libraries easier.

49. Implement Prototype pattern for object cloning.

Answer: Prototype creates new objects by cloning existing instances.

Useful for creating copies of complex configurations.

50. Implement Interpreter pattern for DSL evaluation.

Answer: Interpreter defines grammar representation for language evaluation.

Useful for parsing configurations, query languages, math expressions.

HANDS-ON CODING CHALLENGES

Put your knowledge to the test with these real-world Android development challenges. Each challenge requires implementing multiple design patterns in combination.

Challenge 1: Build a Complete MVVM News App with Repository Pattern

Create a news reader app implementing: • **MVVM architecture** with ViewModel and LiveData/Flow • **Repository pattern** with both remote (API) and local (Room) data sources • **Dependency Injection** using Hilt or manual DI • **UseCase pattern** for business logic (GetNewsUseCase, SearchNewsUseCase) • **Mapper pattern** to convert between DTO, Entity, and Domain models • **Observer pattern** for reactive data updates • **Paging pattern** for infinite scrolling • **Cache-Aside pattern** with expiration logic **Requirements:** - Fetch news from a REST API - Cache articles in Room database - Implement pull-to-refresh - Search functionality with debouncing - Offline support - Loading/Error states - Unit tests for ViewModel and Repository

Challenge 2: Implement Multi-Module E-Commerce App with Clean Architecture

Design and implement a multi-module shopping app: • **Clean Architecture** with Domain, Data, and Presentation layers • **Multi-module structure:** - :core:domain (entities, repositories interfaces, use cases) - :core:data (repository implementations) - :core:network (API service) - :core:database (Room DB) - :feature:products (product listing) - :feature:cart (shopping cart) - :feature:checkout (payment flow) - :app (navigation, DI setup) • **Coordinator pattern** for navigation • **MVI architecture** for state management • **Feature Toggle** for A/B testing checkout flows • **Circuit Breaker** for payment API • **Event Bus** for cross-module communication **Bonus:** - Implement Saga pattern for order creation (create order → process payment → send confirmation) - Add Builder pattern for complex Product objects - Use Strategy pattern for different payment methods

Challenge 3: Create Offline-First Sync System with Event Sourcing

Build a note-taking app with advanced sync: • **Repository pattern** with offline-first approach • **Event Sourcing** to track all note changes • **CQRS** separating read/write operations • **Saga pattern** for conflict resolution during sync • **Command pattern** for undo/redo functionality • **Memento pattern** for saving editor state • **Observer pattern** for real-time updates • **Throttle/Debounce** for auto-save **Features to implement:** - Create, edit, delete notes (all as events) - Automatic sync when online - Conflict resolution (last-write-wins or merge) - Full undo/redo history - Replay all events to rebuild state - Search with caching - Export note history **Technical requirements:** - Store events in Room database - Implement event replay mechanism - Handle network failures gracefully - Show sync status to user

Challenge 4: Build Advanced Image Gallery with Multiple Patterns

Create a feature-rich gallery app: • **Adapter pattern** for RecyclerView • **Decorator pattern** to add filters (Grayscale, Blur, Brightness) • **Proxy pattern** for image caching and lazy loading • **Facade pattern** to simplify image loading library • **Builder pattern** for ImageRequest configuration • **State pattern** for image loading states (Loading, Success, Error) • **Strategy pattern** for different image sources (Gallery, Camera, URL) • **Object Pool pattern** for Bitmap reuse • **Composite pattern** for album hierarchy **Implement:** - Grid view with RecyclerView - Image caching (memory + disk) - Apply multiple filters to images - Save edited images - Create albums and organize photos - Lazy loading with placeholders - Zoom and pan gestures - Memory-efficient Bitmap handling **Advanced:** - Implement LRU cache for images - Handle large images without OOM - Background processing

with WorkManager - Share images to other apps

Challenge 5: Real-Time Chat Application with Reactive Patterns

Build a complete chat system: • **MVVM + MVI hybrid architecture** • **Repository pattern** with WebSocket and Room • **Observer pattern** with Flow and StateFlow • **State pattern** for connection states • **Command pattern** for user actions • **Chain of Responsibility** for message processing • **Mediator pattern** for chat room management • **Visitor pattern** for different message types • **Decorator pattern** for message encryption **Core features:** - Real-time messaging with WebSocket - Offline message queue - Read receipts and typing indicators - Media messages (images, videos) - Message reactions - User presence (online/offline/typing) - Push notifications - Message encryption **Technical implementation:** - Flow for reactive updates - Room for message persistence - WorkManager for sending queued messages - Paging for message history - DiffUtil for efficient list updates **Patterns to apply:** - Use Memento for draft messages - Template Method for message sending flow - Factory for creating different message types - Specification for message filtering

Study Tips & Resources

How to Master Android Design Patterns: 1. **Start Small:** Begin with easy patterns (Singleton, Factory, Observer) before tackling complex ones. 2. **Practice Daily:** Implement one pattern per day in a sample project. 3. **Read Real Code:** Study open-source Android apps to see patterns in action. 4. **Build Projects:** Complete the coding challenges above to solidify your understanding. 5. **Refactor Existing Code:** Take your old projects and apply design patterns to improve them. 6. **Test Your Code:** Design patterns make code more testable - write unit tests! **Recommended Resources:** • Android Developer Documentation (developer.android.com) • "Design Patterns: Elements of Reusable Object-Oriented Software" by Gang of Four • "Clean Architecture" by Robert C. Martin • "Refactoring" by Martin Fowler • Android Architecture Components Guide • Kotlin Coroutines and Flow documentation **Key Principles to Remember:** • SOLID principles • DRY (Don't Repeat Yourself) • KISS (Keep It Simple, Stupid) • YAGNI (You Aren't Gonna Need It) • Composition over Inheritance • Program to interfaces, not implementations **Common Mistakes to Avoid:** • Over-engineering simple solutions • Using patterns without understanding the problem • Ignoring Android-specific constraints (lifecycle, memory) • Not considering testability • Mixing concerns across layers **Interview Preparation:** • Be ready to explain when and why to use each pattern • Understand trade-offs and alternatives • Practice live coding implementations • Know Android-specific applications • Prepare to discuss real projects where you used patterns

Good luck with your Android development journey! ■